Advanced FEM Techniques (AFEMT)

Lecture 1

About the course

- Each session: theory and implementation (deal.ii)
- After first few sessions, one problem per session.
- Why deal.ii? Winner of Wilkinson Prize 2007.
- Assessment: practical project presenting a problem and your deal.ii implementation.

FEM Pipeline (Guiding Map)

PDE $\rightarrow$ weak form $\rightarrow$ discretization $\rightarrow$ assembly $\rightarrow$ solve

Goal: understand how each step appears in deal.ii implementation.

Reference Problem

For first couple of lectures, we use the Poisson problem as running example:

$$-\Delta u = f \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega.$$

In weak form: find $u \in H_0^1(\Omega)$:

$$\int_\Omega \nabla u \cdot \nabla v \, dx = \int_\Omega f \, v \, dx \qquad \forall v \in H_0^1(\Omega).$$

(Conforming) Galerkin method

1. Fix a finite-dimensional subspace $V_h \subset H_0^1(\Omega)$
2. Restrict problem into V_h:

$$\text{Find } u_h \in V_h \ : \ \int_\Omega \nabla u_h \cdot \nabla v \, dx = \int_\Omega f \, v \, dx \qquad \forall v \in V_h.$$

FEM: use continuous piecewise polynomial spaces with respect to a partition (mesh) of the domain.

Mesh and Domain

Let $\Omega \subset \mathbb{R}^d$ be a bounded Lipschitz domain. A mesh is a partition

$$\mathcal{T}_h = \{K\}$$

of Ω into non-overlapping closed, connected, Lipschitz cells (elements) with non-empty interior such that

$$\overline{\Omega} = \bigcup_{K \in \mathcal{T}_h} K.$$

We denote by

$$h_K = \text{diam}\,(K) \qquad h = \max_{K \in \mathcal{T}_h} h_K.$$

Key point: finite element computations carried out cell by cell.

Reference Element and Mapping

A cell K is obtained from a reference cell $\hat{K}$ through a map

$$T_K : \hat{K} \to K, \qquad x = T_K(\hat{x}).$$

Typically:

- $\hat{K}$ unit interval, unit square $[0,1]^d$, unit simplex $\{x \in \mathbb{R}^d : \sum_{i=1}^d x_i \leq 1, \quad \text{and} \quad x_i \geq 0, \forall i = 1, \ldots, d\}$.
- T_K affine or d-linear (deal.ii).

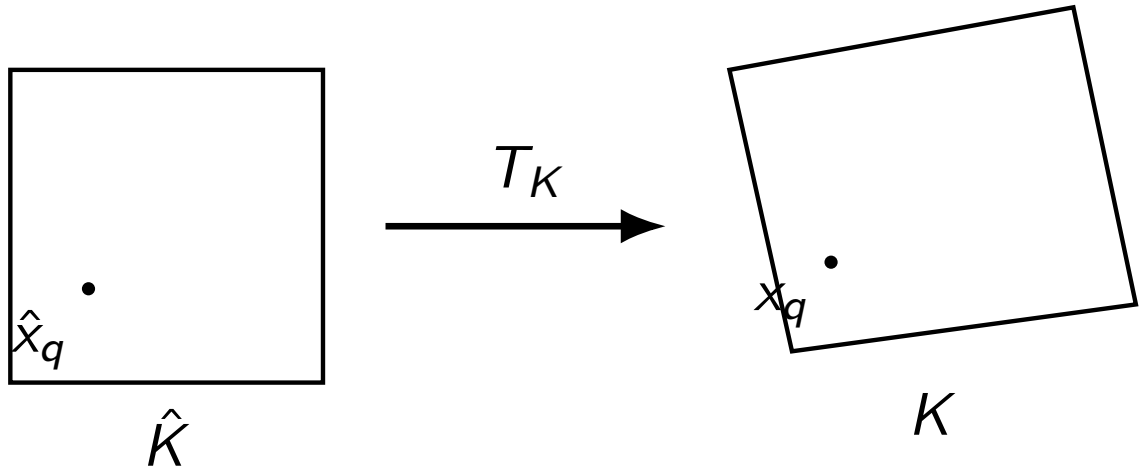
For affine maps,

$$T_K(\hat{x}) = B_K \hat{x} + b_K, \qquad J_K = B_K.$$

Then

$$\int_K f(x)\, dx = \int_{\hat{K}} f\big(T_K(\hat{x})\big) |\det J_K|\, d\hat{x}.$$

Diagram: Reference Cell to Physical Cell



For efficiency basis functions and quadrature rules defined once and for all in the reference cell.

Shape Functions

On each cell we choose a local polynomial space:

- For simplices this is often $\mathbb{P}^k(\hat{K})$ = space of polynomials of total degree up to k;

  `FE_SimplexP< dim, spacedim >`

- For quadrilateral/hexahedral cells deal.II typically uses **tensor-product spaces** $\mathbb{Q}_k(\hat{K})$ = polynomials of degree up to k separately in each variable.
  $\mathbb{Q}_k$ with d-linear maps $\Rightarrow$ optimal approximation

  [Arnold, Boffi, Falk, Math. Comp., 2002]

  `FE_Q< dim, spacedim >`

A basis $\{\phi_i\}$ spans the local approximation space and is tied to the degrees of freedom...

Lagrange Basis

The classical nodal basis is defined by interpolation conditions

$$\phi_i(x_j) = \delta_{ij}.$$

Hence a function is represented by its nodal values.

Advantages:

- intuitive construction,
- simple interpretation of degrees of freedom,
- easy imposition of continuity for conforming methods.

`FE_Q< dim, spacedim >` Gauss-Lobatto nodes

1−D polynomials: Gauss–Legendre vs Gauss–Lobatto

Nodes for Lagrange shape functions and quadrature rules are defined as zeros of certain basis of polynomials...

Gauss–Legendre:

- optimal quadrature points, but endpoints not included,
- Less convenient as interpolation nodes in conforming settings but good for discontinuous Galerkin (DG),
- L^2-orthogonal basis $\rightarrow$ diagonal mass matrix,

Gauss–Lobatto:

- includes the endpoints,
- naturally supports continuity across cell boundaries, BCs
- also attractive for DG (direct knowledge of on boundary)
- Also diagonal mass matrix if Lobatto nodes also used for quadrature ($\rightarrow$ good for time-stepping).

Both give hierarchical and well-conditioned basis.

Quadrature rules supported in deal.ii

`Quadrature<dim>`

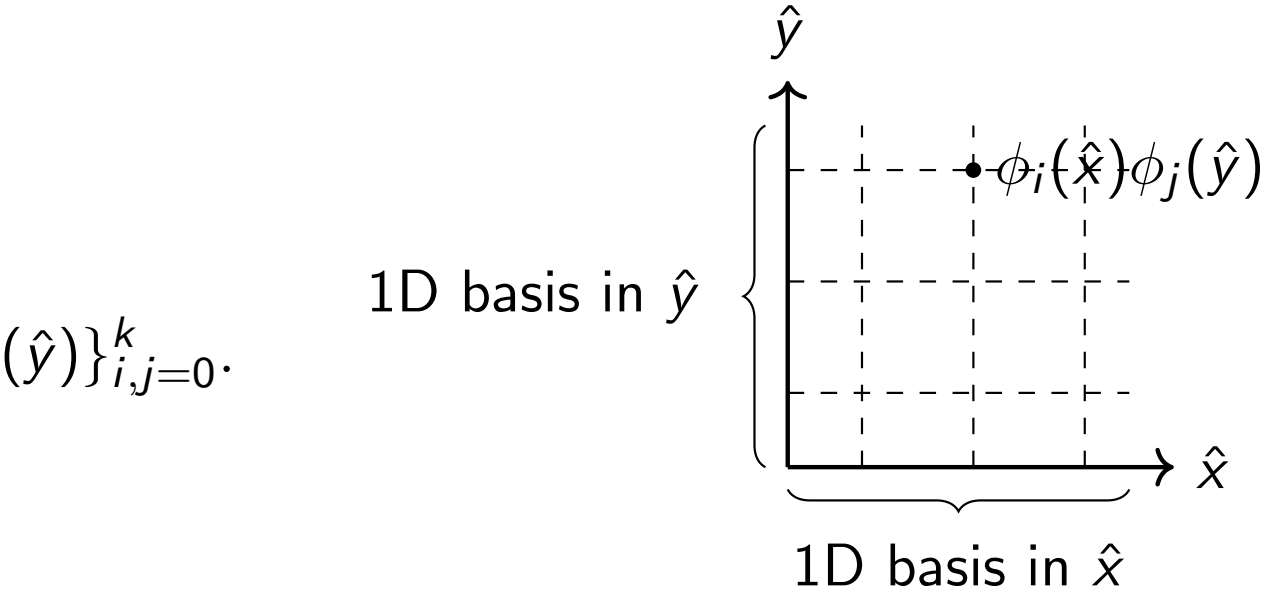
Base class storing quadrature rules (points and weights).
The following derived classes are available:

Quadrature Rule	Description	Polynomial Exactness	Points	Common Usage
`QGauss<dim>`	Gauss-Legendre quadrature	Degree 2n-1	n	Standard cell integration
`QGaussLobatto<dim>`	Gauss-Lobatto quadrature	Degree 2n-3	n	Spectral elements (includes endpoints)
`QMidpoint<dim>`	Midpoint rule	Degree 1	1	Simple approximations
`QTrapezoid<dim>`	Trapezoidal rule	Degree 1	2	Integration with endpoint evaluation
`QSimpson<dim>`	Simpson's rule	Degree 3	3	Higher accuracy with few points
`QMilne<dim>`	Milne's rule	Degree 5	5	Higher order accuracy
`QWeddle<dim>`	Weddle's rule	Degree 7	7	Higher order accuracy

Tensor-Product Basis (deal.II)

On quadrilateral cells,

$$Q_k(\hat{K}) = \mathrm{span}\{\phi_i(\hat{x})\phi_j(\hat{y})\}_{i,j=0}^{k}.$$



In 3D,

$$\phi_{ijk}(\hat{x},\hat{y},\hat{z}) = \phi_i(\hat{x})\phi_j(\hat{y})\phi_k(\hat{z}).$$

This tensor-product structure is one of the main reasons deal.II is so efficient on quad/hex meshes.

Mesh Handling in deal.II (Conceptual View)

- The mesh is stored in a `Triangulation` object
- It represents:
  - cells (elements)
  - faces and edges
  - vertices
  - connectivity (who neighbors whom)
- The mesh is hierarchical (refinement creates parent/child relations)

Key idea: the mesh is an active data structure, not just geometry

Mesh Creation in deal.II

- Built using `GridGenerator`
- Examples:
  - hyper cube
  - hyper ball
  - subdivided domains

```
Triangulation<2> triangulation;
GridGenerator::hyper_cube(triangulation, 0, 1);
triangulation.refine_global(3);
```

Alternative: read meshes from file (`GridIn`)

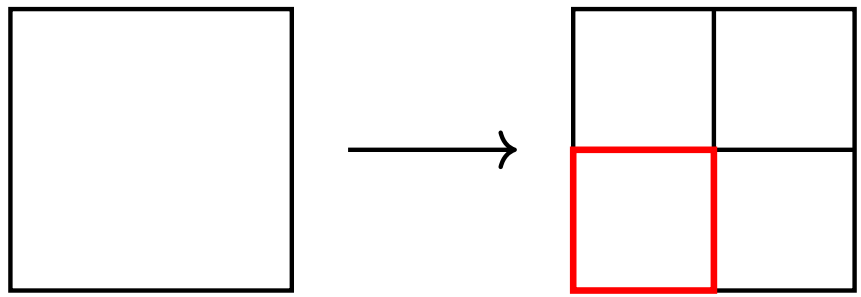
Mesh Refinement (Adaptivity)

- deal.II supports local refinement
- cells can be flagged for refinement/coarsening
- refinement creates a hierarchy of meshes

$$\mathcal{T}_h \rightarrow \mathcal{T}_{h/2}$$

Important: hanging nodes appear and must be constrained

Diagram: Hierarchical Refinement



Refinement is local and hierarchical.